MEA(R)N Full Stack Programming

Objectives:

- In this session, we will learn to…
  - Introduction to NodeJS
  - Use cases of NodeJS
  - Features of NodeJS
  - Architecture of Node JS
  - Parts of Node JS
  - Understand Asynchronous Architecture
  - Applications of Node JS
  - Installation of Node.js on Windows
  - Create a first Node.js application

Introduction to NodeJS

- Node JS is an open-source and cross-platform runtime.
- It is built on Chrome's V8 JavaScript engine for executing JavaScript code outside of a browser.
- It provides an event-driven, non-blocking (asynchronous) I/O and cross-platform runtime environment for building highly scalable server-side applications using JavaScript.
- It is asynchronous architecture or non-blocking in nature.
- It enabled to build Real-Time Chats, Complex Single-Page applications, Real-time collaboration tools, Streaming apps, JSON APIs based application.
- It runs on various platforms (Windows, Linux, Unix, Mac OS X, etc.)

Node JS Uses Cases

1. RESTFul APIs and Microservices

- Easily handles multiple requests concurrently
- Common for backend APIs in mobile & web apps
- Lightweight microservice architecture

2. Real Time Application

- Chat Application
- Live notification
- Online collaboration tools such Google docs App
- Multiplayer gaming

3. SPA Backend

- Angular, React, Vue front end offen pair with Node JS
- JSON based communication fits naturally with JS

Node JS Uses Cases

4. Streaming Application

- Media streaming services (Audio / Video)
- File upload / Download
- Real-Time data processing (IoT dashboard)

5. IoT Application

- Works well with sensor-based echo systems
- Handled thousand of lightweight connections efficiently.

6. SPA Backend

- Angular, React, Vue front end offen pair with Node JS
- JSON based communication fits naturally with JS

Node JS Uses Cases

7. **Proxy & API Gateway**
   - Act as a middleware to manage backend services
   - Integrating external APIs

8. **Serverless Computing**
   - Widely used on AWS Lambda, Azure functions, GCP cloud Functions
   - Scalable deployment for event-based triggers

9. **Command-line tools and Developer tools**
   - NPM echo system provides strong package support.
   - Build tools, bundles, and generators like webpack

Features of NodeJS

Highly Scalable

Easily scalable the application in both horizontal and vertical directions.

Single threaded

Node.js follows a single threaded model with event looping.

Non-blocking I/O

It enables the platform to manage several requests concurrently without delaying other ones. With the help of this capability, Node.js apps can handle heavy traffic and data processing without experiencing any lag or crashes.

Open source

Node.js has an open-source community which has produced many excellent modules to add additional capabilities to Node.js applications.

Features of NodeJS

Caching techniques

It provides the caching of a single module. Whenever there is any request for the first module, it gets cached in the application memory, so you don’t need to re-execute the code.

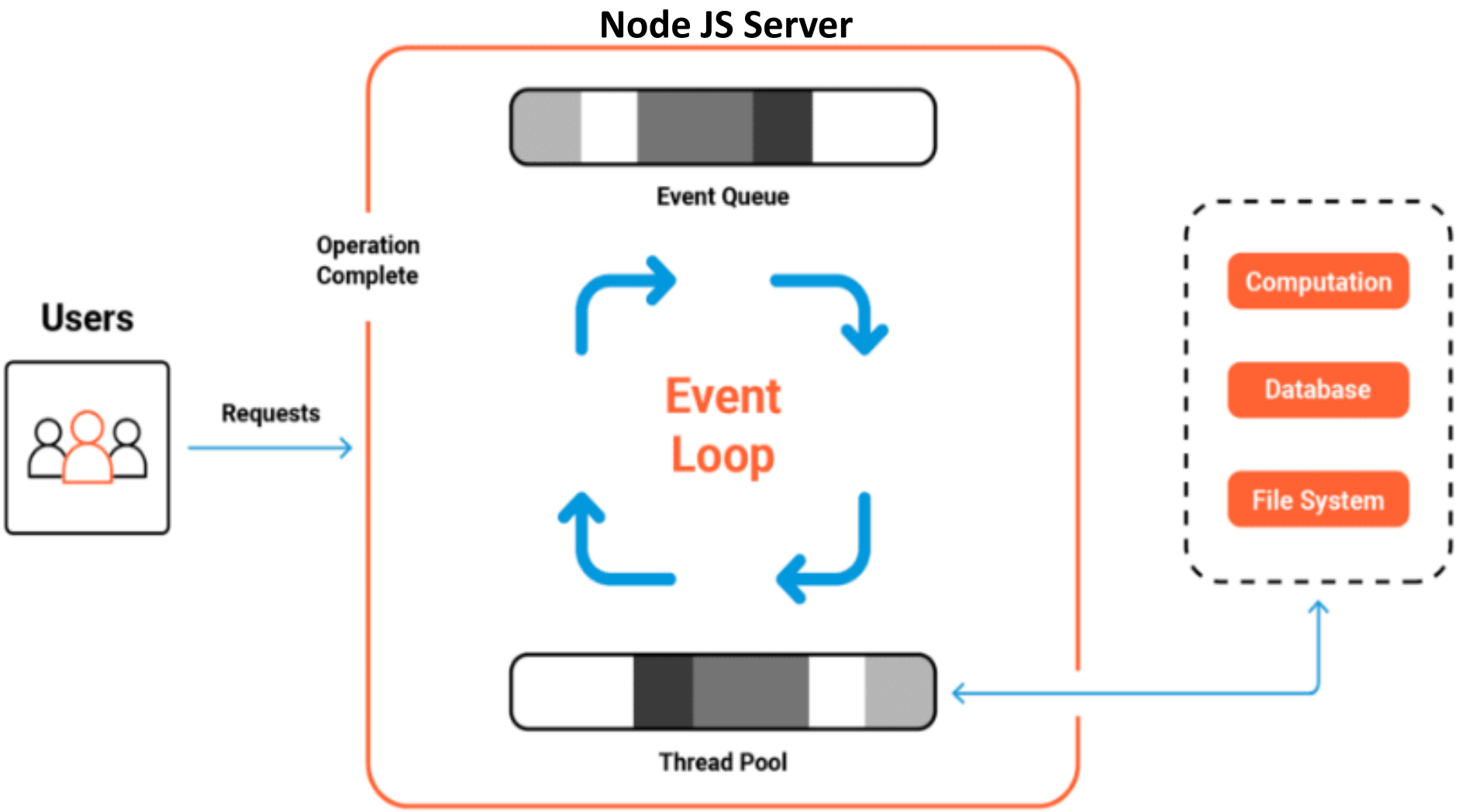
Data Streaming

NodeJS HTTP request and response are considered as two separate events.

Cross-Platform Support

It is simple to execute an application on a range of servers and environments.

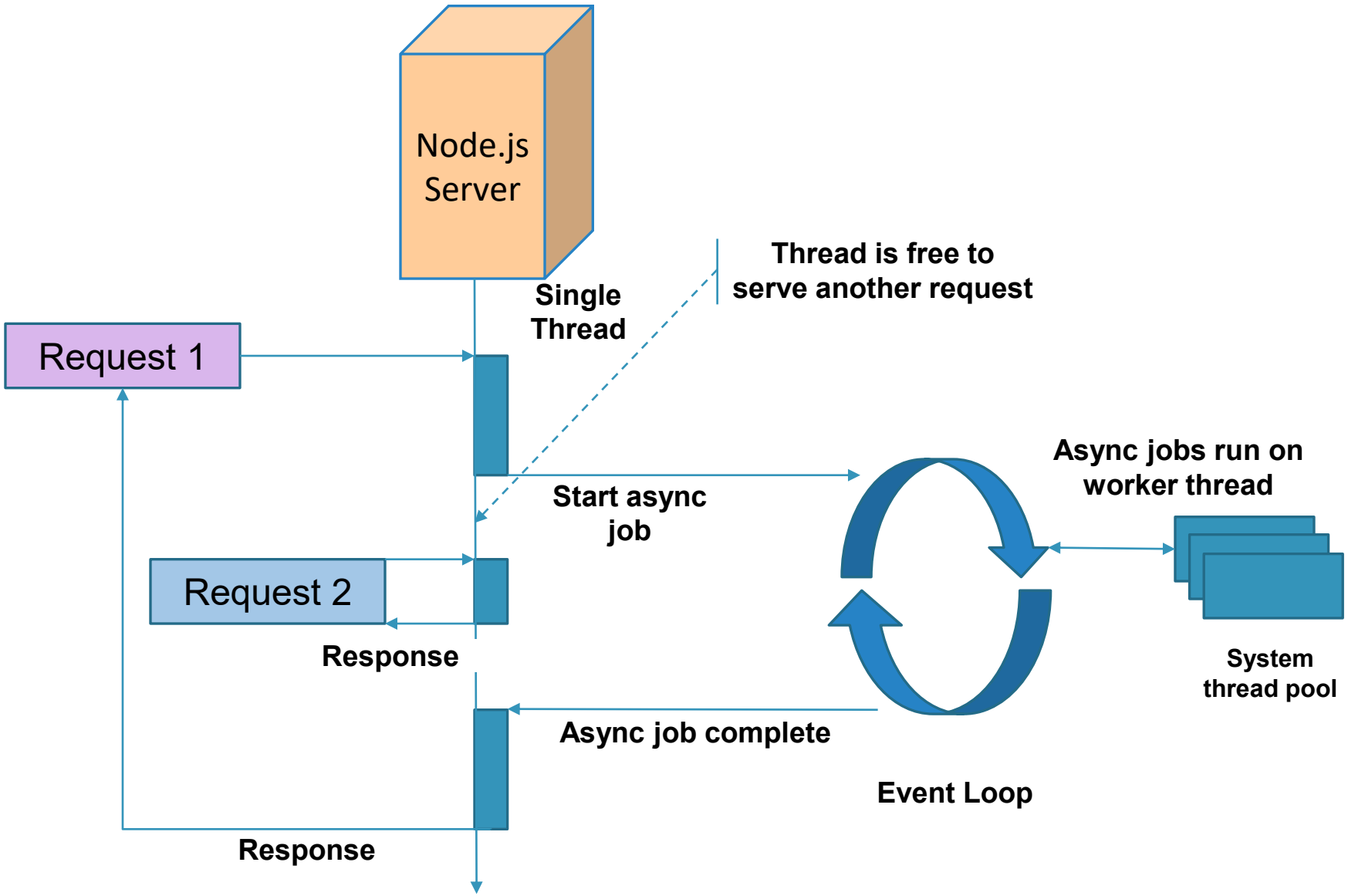
Architecture of Node JS



Parts of Node JS

1	Node JS Server	Node.js server is a server-side platform that takes requests from users, processes those requests, and returns responses to the corresponding users
2	Event Queue	Event Queue in a Node.js server stores incoming client requests and passes those requests one-by-one into the Event Loop
3	Event Loop	Event Loop indefinitely receives requests and processes them, and then returns the responses to corresponding clients
4	Thread Pool	Thread pool consists of all the threads available for carrying out some tasks that might be required to fulfill client requests
5	External Resources	External resources are required to deal with blocking client requests. These resources can be for computation, data storage, etc.

Asynchronous Architecture



How a Request is Processed in an Asynchronous Environment

1. **Request 1** starts executing on the main thread (Event Loop) of the Node.js server.
2. During processing, if **Request 1** requires interaction with external resources (***e.g., file system, database, network API***), the main thread delegates this work to background workers or system services.
3. **Request 1** is then moved off the main thread while waiting for the async operation to complete.
4. This frees the main thread to immediately accept and process **Request 2** or other incoming requests.

How a Request is Processed in an Asynchronous Environment (Contd.)

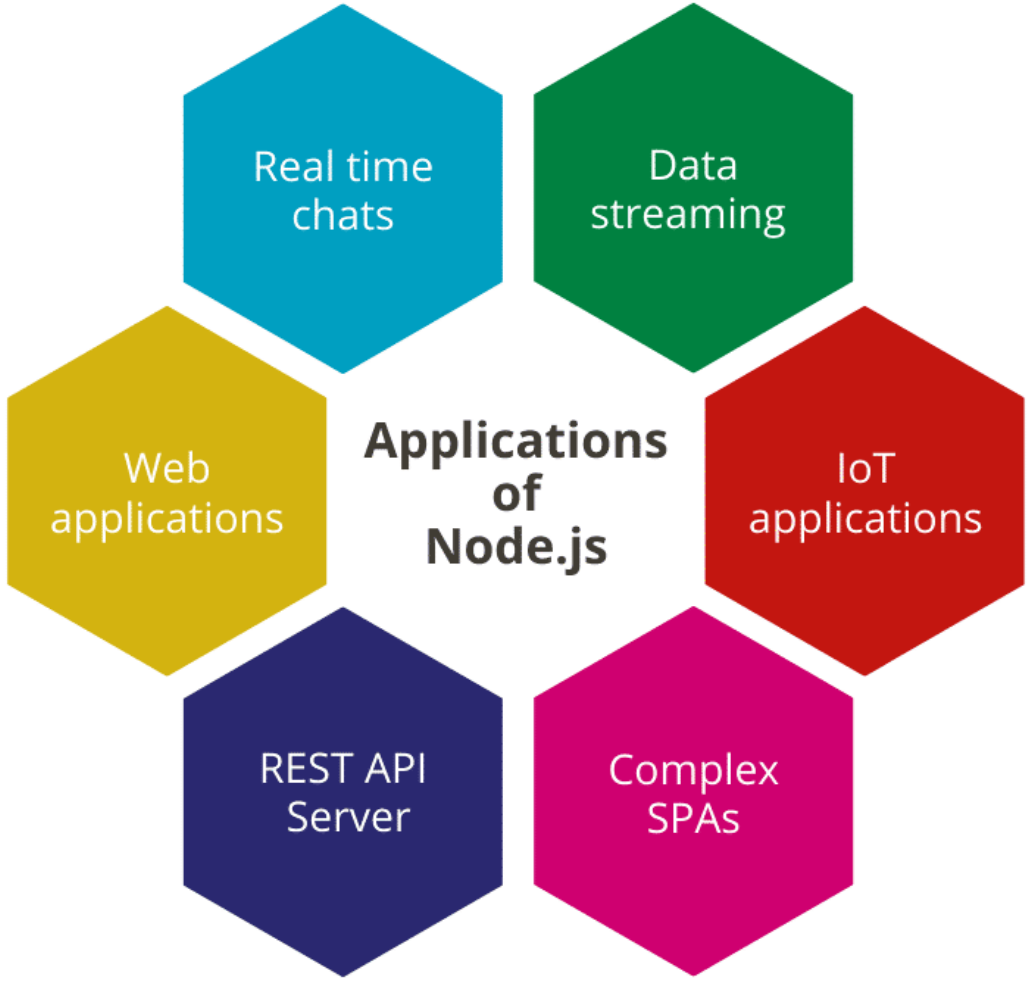
5. When the external operation for **Request 1** finishes, it triggers an event or callback, notifying the Event Loop.
6. The Event Loop then executes the corresponding callback/event-handler for **Request 1**.
7. The event-handler continues processing **Request 1** and eventually sends the response back to the client.
8. Until the event is received, **Request 1** remains pending without blocking the main thread.

Node vs. other server-side platforms

Here is a table to understand the key differences between Node.js and other server-side platforms:

Platform	Language	Model	Performance
Node JS	JavaScript	Non-blocking IO	Fast
PHP	PHP	Process Based	Moderate
Ruby on Rails	Ruby	Multi-thread	Moderate
Java	Java	Process Based	Fast

Applications of NodeJS



Installation of Node.js on Windows

- **Step-1**
  The first step to install Node.js on windows is to download the installer. Visit the official Node.js website i.e) **https://nodejs.org/en/download/** and download the .msi file according to your system environment (32/64-bit).

- **Step-2**
  Follow the installation steps and finish it.

- **Step-3**
  Verify the Node.js was properly installed or not.
  node -v
  - *If node.js was completely installed on your system, the command prompt will print the version of the node.js installed.*
  - *If you get a message back saying node was not found, then add it to the path manually in environment.*

Example

- **Create a file name with app.js and add below code.**

```
var http = require('http’);
http.createServer(function (req, res) {
    res.writeHead(200, {'Content-Type': 'text/html'});
    res.end('Hello World!');
}).listen(3000);
```

- **Now type the below command on prompt :** node app.js
- **Then, open a chrome browser and type the below URL.**

  http://localhost:3000
- **It will print following output in browser.**

  Hello World!

What is REPL ?

- REPL stands….

  Read – It reads the JavaScript code you input

  Eval – It execute the code

  Print – It print the result to the console

  Loop – It loops back to beginning, waiting for the next input

- **Example (Start command prompt, Type node)**

```
> 10 * 10                                  // 100
> let name = "Parul University"            //Parul University
> function sum(num1, num2) {
      return num1+num2;
  }
> sum(10,10)                               // 30
```

Objectives:

- In this session, we will learn to…
  - Working with modules

Working with modules

- A **module** in Node.js is a **self-contained block of code** that exists in its own file and performs a specific functionality.
- Node.js automatically treats **every JavaScript file as a module**, allowing separation of logic and reusability across the application.
- Modules can be a single file or a collection of multiple files / folder.
- Each module has its own scope, meaning variables or functions defined in a module **cannot be accessed outside** unless explicitly exported.
- It enables to break down a complex piece of code into manageable chunks.
- There are three types of modules.
  1. Core modules
  2. Local modules
  3. Third Party Modules

Types of module:

Core modules

- Core modules are built directly into Node.js and come bundled with your Node.js installation.
- No need to install anything extra to use them.
- It provide fundamental functionality that most Node.js applications need.
- Local modules such as **fs, http, path, os, events, and crypto** etc.
- **Module** can be loaded into the program by using **require** function.
- **Example :** const http = require('http');

Types of module:

Local modules

- **Local modules** are the modules you create yourself to organize your own application's code.
- These are the files you write that contain your application-specific logic and business rules.
- Unlike built-in and external modules, local modules are created locally in your Node.js application.
- For example, when you create a file called ***database.js*** that contains your database connection logic, ***authentication.js*** that contains to handle authentication logic etc.

```
const connection = require('./config/database.js');
```

Types of module:

Third party modules

- Third-party modules are modules created by other developers and shared with the community through the Node Package Manager, commonly known as npm.
- The npm ecosystem is one of the largest software registries in the world, containing hundreds of thousands of packages.
- Some of the popular third-party modules are **Mongoose, express, angular, and react**
- **For example :** const express = require('express');

Why modules are important in Node JS ?

- Instead of writing everything in one large file, modules help break an application into smaller, logical parts.
- Functions and logic written once in a module can be reused throughout the application — no need to rewrite code.
- When code is divided into small, focused modules, it becomes easier to: Understand, Debug, Modify, and Extend
- Modules expose only what is necessary using exports.
- Different developers can work on separate modules **in parallel** without conflicts.
- Node.js supports installing thousands of pre-built third-party modules through npm.

NPM (Node Package Manager)

- NPM (Node Package Manager) is the default package manager for Node.js and is written entirely in Javascript.
- NPM manages all the packages and modules for Node.js and consists of command line client npm.
- A package contains all the files needed for a module and modules are the JavaScript libraries that can be included in Node project according to the requirement of the project.
- NPM can install all the dependencies of a project through the **package.json** file.
- It can also update and uninstall packages.

NPM Modules

Install a Module

- The most required NPM modules or packages would already be included when you install Node.
- In case you need to install a module, use the command:

  npm install (with no args, in package dir)
- After installing a module, you can include it using the require()function.
- By default, packages/modules are installed in local mode. If you want to install in global mode, use the –g option:

  npm install (with no args,in package dir)-g

TIP: When installing a module in local mode, the modules are installed in the node_modules subdirectory.

NPM Modules

Uninstall a Module

- To uninstall a module, use the command:

  npm uninstall <Module Name>

Update a Module

- To update a module, you need to update the package.json file. Use the command:

  npm update <Module Name>

Search a Module

- To search for a module in the online repository, use the command:

  npm search <Module/Package Name>

Synchronous vs Asynchronous Operations

Synchronous Operations:

Characteristics of synchronous operations:

- Block the entire Node.js process until the operation completes
- Easier to write and understand because code executes in order.
- Method names end with Sync (like readFileSync, writeFileSync).
- Return values directly instead of using callbacks.
- Not recommended in production, unless used only at startup.

Example:

```
const fs = require('fs’);
const data = fs.readFileSync('example.txt', 'utf8’);
console.log(data);
```

Asynchronous Operations

- Asynchronous process is also called non-blocking code.
- It allows a program to **continue execution of other code** while waiting for the **heavy-running tasks** to complete **without freezing the program**.
- Node.js runs single-threaded JavaScript code in the **event loop** and **offers a background Worker Pool** to handle **heavy operations** such as file I/O.
- Enables helps to make the Node.js application scalable.
- Enables the system to execute multiple tasks by making use of the waiting time between I/O operations
- Allows a server to maintain many concurrent connections

Asynchronous Operation in Node – Part-1

Let's consider a file-read operation in node.

- **The following is a code in node to read a file synchronously.**

```
var inputfile = require ('fs');
var data = inputfile.readFileSync('DATA', 'utf8'); //blocks here
console.log(data);
console.log('Done!');
```

- **Output**

```
node filesync.js
File starts here!
This is a test file with text.
One of the main features of Node.js is the event-based asynchronous architecture. In traditional programming practice, most I/O o
perations happen synchronously, that is, the system waits for any inputs to be received before moving ahead and completes each ta
sk before proceeding to the next.

File ends here!
Done!
```

Asynchronous Programming in Node – Part-2

- **The following is a code in node to read a file asynchronously.**

```
var inputfile = require ('fs');
inputfile.readFile('DATA', 'utf8', function(err, data){
    console.log(data);
});
console.log('Done!');
```

- **Output**

```
node fileasync.js
Done!
File starts here!
This is a test file with text.
One of the main features of Node.js is the event-based asynchronous architecture. In traditional programming practice, most I/O o
perations happen synchronously, that is, the system waits for any inputs to be received before moving ahead and completes each ta
sk before proceeding to the next.

File ends here!
```

What is the Difference ?

Part-1

- The system waited for the file to be read before printing output to the console.

Part-2

- The system did not wait and moved to execute the next statement and print “Done!”

How does Node implement asynchronous architecture ?

- Node uses three concepts to implement asynchronous architecture in JavaScript:

  1. *Callback functions*
  2. *Promises*
  3. *Async/ Await*

Callback functions

- Is the asynchronous equivalent of a function
- Is defined as a function that is passed on to another function as its argument

```
inputfile.readFile('DATA', 'utf8', function(err, data) {
    if(err) {
        console.error(err);
        return;
    }
    console.log(data);
});
```

- **function(err, data){}** is the callback function in the above code.
- Node makes heavy use of callback functions, and all APIs have been written in a way that supports callbacks.

Promises

- Using these promise-based API methods can be helpful for performing asynchronous programming since they also run asynchronously in a non-blocking manner

```
const fs = require('fs');
fs.promises.readFile('hello.txt', 'utf8’)
    .then(data => {
        console.log(data);
    })
    .catch(err => {
        console.error(err);
    });
```

Async/ Await

- Using these promise-based API methods with Async/ Await is a newer concept.

```
const fs = require('fs');
async function readFileAsync() {
    try {
        const data = await fs.promises.readFile('hello.txt', 'utf8');
        console.log(data);
    } catch(err) {
        console.error(err);
    }
}
readFileAsync();
```

Objectives:

- In this session, we will learn to…
  - Working with core modules.

Core Modules

FS module

- The **File System (fs) module** is a **core module** in Node.js that enables interaction with the **file system of the operating system**.
- It allows a Node application to perform **creating, reading, writing, updating, deleting, copying** files and directories.
- It also supports **advanced operations** like streaming and watching file changes.
- **Example**

```
const fs = require('fs');

const fs = require('fs').promisies
```

FS module - File Handling Methods

fs.open()	Opens a file and returns a file descriptor.
fs.close()	Closes an open file descriptor.
fs.read()	Reads data from a file using file descriptor.
fs.write()	Writes data to a file using file descriptor.
fs.readFile()	Reads the entire content of a file asynchronously.
fs.readFileSync()	Reads file content synchronously (blocking).
fs.writeFile()	Creates a new file or overwrites an existing file.
fs.writeFileSync()	Synchronous version of writeFile().
fs.appendFile()	Appends data to a file without deleting existing content.
fs.appendFileSync()	Synchronous version of appendFile().

FS module - File Handling Methods

fs.unlink()	Delete a file
fs.unlinkSync()	Synchronous file deletion.
fs.rename()	Renames or moves a file.
fs.renameSync()	Synchronous rename.
fs.copyFile()	Copies one file to another.
fs.copyFileSync()	Synchronous file copy.

FS module – Directory Handling Methods

fs.mkdir()	Creates a new directory.
fs.mkdirSync()	Synchronous folder creation.
fs.rmdir()	Removes a directory (older method).
fs.rm()	Deletes files or folders (modern method).
fs.readdir()	Reads the contents of a directory.
fs.readdirSync()	Synchronous directory read.
fs.existsSync()	Checks if a file or directory exists.
fs.access()	Checks file permissions.
fs.stat()	Returns detailed file or folder information.

Core Modules

OS module

- The **OS (Operating System) module** in Node.js is a **built-in core module** that provides utilities and methods to interact with the underlying operating system.
- It allows Node.js applications to fetch information about the system environment — such as the CPU, memory, hostname, network interfaces, platform, etc.
- **Example**

```
const os = require('os’);
console.log('Operating System:', os.type());
console.log('Platform:', os.platform());
console.log('Release:', os.release());
```

Core Modules

Path module

- **Path module** in Node.js is a **core (built-in) module** that provides a set of tools and functions for working with **file and directory paths**.
- It ensures that paths are handled in a **platform-independent** manner, meaning your code will work correctly on **Windows, macOS, and Linux** without needing to worry about differences in path formats.
- **Example**

```
const filePath = path.join(__dirname, 'data', 'info.txt’);
console.log('Full Path:', filePath);
console.log('Directory:', path.dirname(filePath));
console.log('File Name:', path.basename(filePath));
console.log('Extension:', path.extname(filePath));
```

Core Modules

Path module – More methods

1. path.basename() - Get file name
2. path.dirname() - Get directory name
3. path.extname() - Get file extension
4. path.join() - join paths safely
5. path.resolve() -Convert relative -> absolute path
6. path.parse() -Split path into components
7. path.format() -Create path from parts
8. path.isAbolute() -Check if path is absolute
9. path.sep -Get OS-specific separator

Understanding the MATH (Math Module)

- Math.js is an extensive math library in Node.js.
- It features a flexible expression parser with support for symbolic computation, comes with a large set of built-in functions and constants.
- It offers an integrated solution to work with different data types like numbers, big numbers, complex numbers, fractions, units, and matrices.
- Powerful and easy to use.

Understanding the MATH (Math Module)

```
const math = require('math')

console.log('functions and constants')
console.log(math.round(2312.3433, 3))          // 2312
console.log(math.atan2(3, -3) / math.PI)       // 0.75
console.log(math.log(10000, 10))               // 9.210340371976184
console.log(math.sqrt(4))                      // 2
console.log(math.pow(2,2))                     // 4
console.log(math.cos(10))                      //-0.8390715290764524
console.log(math.ceil(3456.67))                //3457
```

Understanding the MATH (Math Module)

```
const math = require('math')

console.log(math.floor(3456.67))          // 3456
console.log(math.exp(3))                  //20.085536923187668
console.log(math.max(10,20,4,5,2))        //20
console.log(math.min(10,20,4,5,2))        //2
console.log(math.random(10))              //0.9271342868429719
console.log(math.trunc(2312.3433, 3))     s//2312
```

Objectives:

- In this session, our will learn to…
  - Introduction to Express.JS
  - Features of Express.JS
  - Working with Middleware
  - Discuss different method to implement middleware
  - Working with Routing
  - Features of Routing in Express.js

What is Express ?

- Express.js is a web application framework that is built on top of Node.js.
- It provides a minimal interface with all the tools required to build a web application.
- It adds flexibility to an application with a huge range of modules available on npm that you can directly plug into Express as per requirement.
- It makes it easier to organize your application's functionality with middleware and routing.
- It adds helpful utilities to Node.js HTTP objects and facilitates the rendering of dynamic HTTP objects.

Key features of Express.JS

Routing	Middleware	Templating Engines
Easily define routes to handle different HTTP methods and request paths	Utilize middleware functions to handle tasks during the request-response cycle, enhancing flexibility.	Support for various templating engines to dynamically generate HTML content.

Use cases of Express.JS

Building web applications and APIs

Single-page applications (SPAs).

Real-time applications using technologies like Socket.io.

Microservices architecture.

Installing Express

- To install Express.JS, first you need to create a project directory and create **package.json** which will be holding the project dependencies, use the below code to perform the same:

  npm install

- Now, you can install the Express.JS package in your system, to install it globally, use the below code:

  npm install -g express

- Or, if you want to install it locally into your project folder, you need to execute the below command:

  npm install express --save

Example

- Create a file name with app.js and add below code.

```
const express = require('express');
const app=express();
app.get('/',(req,res)=>{
    res.send("Welcome to Express"); });

app.listen(3000,()=>{
    console.log("listening to port 3000"); });
```

- Now type the below command on prompt.

  node app.js
- Then, open a chrome browser and type the below URL.

  http://localhost:3000
- It will print following output in browser.

  Hello World!

Working with Routing

- Routing refers to the process of determining a specific behavior of an application.
- It is used for defining how an application should respond to a client request to a particular route, path or URI along with a particular HTTP Request method.
- Each route can contain more than one handler functions, which is executed when the user browses for a specific route.
- There are four main HTTP methods that can be supplied within the request (such as GET, POST, PUT, DELETE) and URL patterns to specific pieces of code that handle those requests.
- Example : app.METHOD(PATH, HANDLER)

Basic Routing Handling

```
var express = require('express');
const app = express();
app.use(express.json());
//Landing page
app.get('/', function (req, res) {
    console.log(‘root request received !’);
    res.send(‘<h2 style= "font-family: arial; "> Welcome to Web </h2>');
});
//Add new user
app.post('/adduser', function (req, res) {
    console.log(‘Add new user – POST request recieved!’);
    res.send(‘<h2 style= "font-family: arial; "> Add new user - POST </h2>');
})
```

Basic Routing Handling

```
//Edit existing user
app.put('/edituser', function (req, res) {
    console.log('Edit user request received !');
    res.send('<h2 style= "font-family: arial; "> Edit user</h2>');
});
//Delete user
app.delete('/deluser', function (req, res) {
    console.log('Delete user request received !');
    res.send('<h2 style= "font-family: arial; "> Delete user</h2>');
})

const port = process.env.PORT || 8080;
app.listen(port, () => console.log(`Listening on port ${port}..`));
```

Features of Routing in Express.js

Route Parameters

Capture values from the URL using parameters

```
app.get('/users/:userId', (req, res) => {
    const userId = req.params.userId;
    res.send(`User ID: ${userId}`);
});
```

Features of Routing in Express.js

Query Parameters

Access query parameters from the URL

```
app.get('/search', (req, res) => {
    const query = req.query.id;
    res.send(`Search Query: ${query}`);
});
```

Features of Routing in Express.js

Route Middleware

Apply middleware functions to specific routes or globally using app.use()

```
const middleFunction =
    (req, res, next) => {
        // Middleware logic
        next();
    };
    app.get('/protected', middleFunction,
        (req, res) => {
        res.send('Protected Route');
});
```

Features of Routing in Express.js

Router Objects

Organize routes into modular routers using express.Router()

```
const router = express.Router();

router.get('/', (req, res) => {
  // Handle GET request to /
});


app.use('/api', router);
// Mount the router at /api
```

Features of Routing in Express.js

Route Handlers and Middleware Chains

Chain multiple route handlers or middleware functions for a single route.

```
app.get( '/example’,  (req, res, next) => {
   // First middleware
   next();
 },
 (req, res) => {
   // Second middleware or route handler
   res.send('Example Route');
 }
);
```

Features of Routing in Express.js

Express Router Module

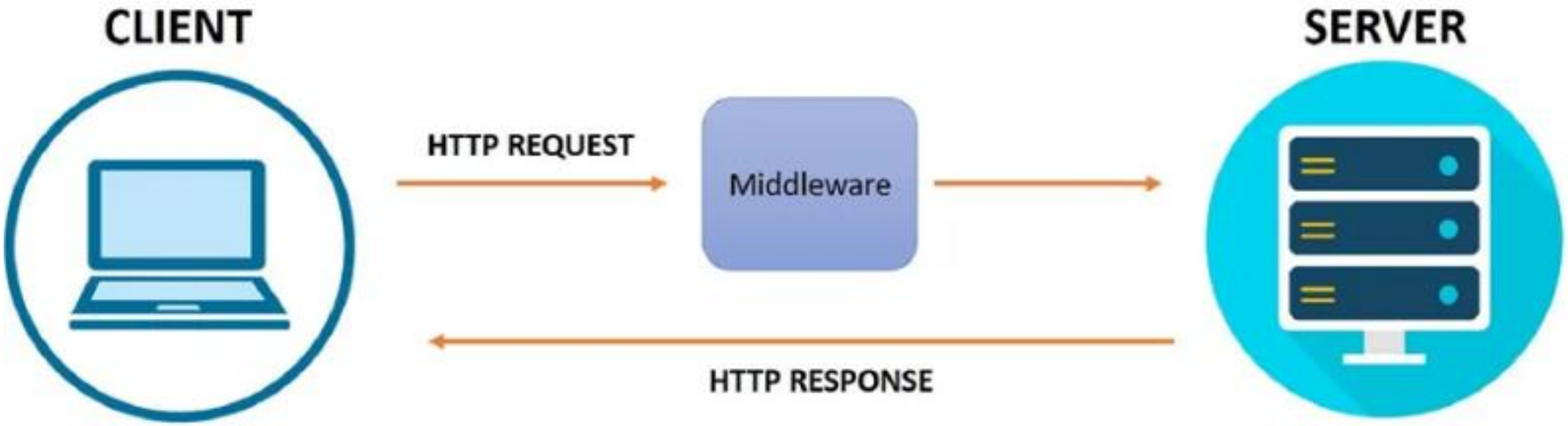
Use the Express Router module for more complex applications to separate concerns and organize routes.

```
// routes/user.js
const express = require('express');
const router = express.Router();
router.get('/:userId', (req, res) => {
  // Handle GET request to /user/:userId
});
module.exports = router;
// app.js
const userRouter =
require('./routes/user');
app.use('/user', userRouter);
```

Working with Middleware

- It plays a crucial role in handling tasks during the request-response cycle.
- Middleware gets executed after the server receives the request and before the controller actions send the response.
- Middleware has the access to the request object, responses object, and next, it can process the request before the server send a response.
- Middleware functions can perform various tasks, such as modifying request and response objects, terminating the request-response cycle, and calling the next middleware function.

Working with Middleware



Working with Middleware

There are different types o Middleware used in the Express.

1. Application-level Middleware
2. Router-level Middleware
3. Error - Handling Middleware
4. Built-ins Middleware
5. Third Party Middleware
6. Method Specific Middleware
7. Multiple Callback Middleware

Types of Middleware

1. **Application-level Middleware**

- This middleware is executed for every incoming request to the application.
- It logs a message to the console and then calls the **next()** function to pass control to the next middleware in the stack.

```
app.use((req, res, next) => {
    console.log('This is an application-
        level middleware');
    next();
});
```

Types of Middleware

2. **Router-level Middleware**

- Similar to application-level middleware, but it's applied to a specific router.
- In this example, it logs a message for every request to the router.

```
const router = express.Router();

router.use((req, res, next) => {
    console.log('This is a router-level
            middleware');
    next();
});
```

Types of Middleware

3. Error - Handling Middleware

- This middleware has four parameters, with the first one (err) representing an error.
- It's executed when an error occurs during the request-response cycle. It logs the error and sends an error response.

```
app.use((err, req, res, next) => {
    console.error(err.stack);
    res.status(500).send('Something
            went wrong!');
});
```

Types of Middleware

4. **Built-in Middleware**

- Express provides built-in middleware functions for common tasks.
- The above example demonstrate JSON and form data parsing, as well as serving static files.

```
app.use(express.json());
app.use(express.urlencoded(
    {
      extended: true
    })
);
app.use(express.static('public'));
```

Types of Middleware

5. **Third-party Middleware**

- Third-party middleware modules like **body-parser, helmet, and morgan** enhance Express functionality.
- They are added to the application using app.use().

```
const bodyParser = require('body-parser');
const helmet = require('helmet');
const morgan = require('morgan');
app.use(bodyParser.json());
app.use(helmet());
app.use(morgan('combined'));
```

Types of Middleware

6. **Method-specific Middleware**

- Middleware can be applied to specific HTTP methods and routes. In this example, the middleware is executed only for GET requests to the '/route' path

```
app.get('/route', (req, res, next) => {
    console.log('This middleware is specific to the GET method for /route');
    next();
});
```

Types of Middleware

7. Multiple Callback Middleware

- Multiple middleware functions can be chained together.
- In this example, middleware1 and middleware2 are executed sequentially before reaching the route handler

```
app.get('/route',
    middleware1, middleware2,
    (req, res) => {
        // Route handler
        res.send('Response from the
            route handler');
});
```

Objectives:

- In this session, our will learn to…
  - Introduction to Template Engine.
  - Working with Template Engine.

Introduction to Template Engines

- Template Engines are essential tools in web development that allow you to generate dynamic HTML content by combining templates with data.
- A Template Engine is a technology that enables the separation of HTML markup and dynamic data.
- It simplifies the process of rendering dynamic content by providing placeholders for data within your HTML templates.
- Easily bind dynamic data to templates for rendering.
- It allow you to break down complex UI components into manageable parts.
- This separation enhances code readability, reusability, and maintainability.

Types of Template Engines

Numerous template engines are compatible with Express, the most popular of which include:

- EJS (Embedded JavaScript)

  Allows you to embed plain JavaScript within HTML-like tags. It is known for having a familiar and easy-to-learn syntax.

- Handlebars

  A simple and powerful engine with a syntax resembling HTML.

- Pug (formerly known as Jade)

  It offers a concise and expressive syntax.

Types of Template Engines (Contd.)

- Mustache

  It is Similar to Handlebars, it is a logic-less template engine that works well in both server-side and client-side environments.

- Nunjucks

  A powerful and feature-rich templating engine inspired by Jinja2, known for its flexibility and power.

- Liquid

  A cross-language template engine with a Ruby origin but with various JavaScript ports available.

Working with Template Engines

Install express and a Template Engine.

npm install express, **ejs**

Configuring Express to Use the Chosen Template Engine

```
app.set('views',path.join(__dirname,'views'));
app.set('view-engine', 'ejs’)

app.get('/', (req, res) => {
   res.render("index.ejs", { name: ‘Parul University’})
})
```

Index.ejs file

```
<h1>Welcome, <%= name %></h1>
```

Objectives:

- In this session, our will learn to…
  - Error handing and logging.

Error handling and logging

- Error handling and logging are crucial aspects of building robust and maintainable Node.js and Express.js applications.
- Error handling ensures that your application can gracefully handle unexpected issues and recover from errors, while logging provides valuable information for debugging and monitoring.
- Errors thrown in Node.js applications can be handled in the following ways:
  - try-catch block, promises and promise callbacks, and Event Emitter
- Logging involves recording information about the application's runtime behavior in a more persistent manner.
- There are common levels of logging in Node.js:
  - Error, warn, info, debug

Error handling in Node.JS

Using try-catch block

- Use try-catch blocks to handle synchronous errors within your code.

  try statement - test a block of code to check for errors.

  catch statement - handle the error if any are present.

  throw statement - make your own errors.

  finally statement - it runs regardless of the result of the try-catch block.
- Syntax

```
try {
    Try Block to check for errors.}
catch(err) {
    Catch Block to display errors.}
```

Error handling in Node.JS

Using try-catch block

Example

```
let divideSync = function (x, y) {
    if (y === 0) {
        return new Error("Can't divide by zero")
    } else {
        return x / y
    }
}
let result = divideSync(9, 3)
```

Error handling in Node.JS

Promise

- Promise are easy to manage when dealing with multiple asynchronous operations where callbacks can create callback hell leading to unmanageable code.
- Promises are used to handle asynchronous operations
- **Syntax**

```
.then(function(result){
        //handle success
}, function(error){
      //handle error
})
```

Error handling in Node.JS

Promise

Example

```
let promise = new Promise(function (resolve, reject)  {
    reject('Promise Rejected’)})

promise
  .then(function (successMessage) { console.log(successMessage); })
  .catch(function (errorMessage) {
    //error handler function is invoked
    console.log(errorMessage);
  });
```

Event Emitter

- Event emitters are objects in Node.js that trigger an event by sending a message to signal that an action was completed.
- Events are composed of an identifying string and any data that needs to be passed to the listeners.
- It is a Publisher / Subscriber pattern.
  - Publish – Send a message (event).
  - Subscriber – It received the event and perform the action.

Error handling in Node.JS

Event Emitter

Example

```
const EventEmitter = require('events’); // Importing events
var eventEmitter = new EventEmitter(); // Initializing event emitter instances

// Registering to myEvent
eventEmitter.on('myEvent', (msg) => {
  console.log(msg);
});

// Triggering myEvent
eventEmitter.emit('myEvent', "First event");
```

Error handling in Express.JS

Using express middleware

- It is a process of capturing and responding to errors that occur during the execution of an Express application.
- You can handle errors using middleware functions, which are functions that have access to the request and response objects, as well as the next middleware function in the application's request-response cycle.
- Example

```
const handlername = (err, req, res, next) => {
    //Code to be execute
}
```

Error handling in Express.JS

Using express middleware - Example

```
const errorHandler = (err, req, res, next) => {
    console.error(err.stack);
    res.status(500).send('Something went wrong!');
};

// Use the custom error handling middleware
app.use(errorHandler);

app.listen(3000, () => {
    console.log('App is listening on port 3000');
});
```

Error handling in Express.JS

Async/ await middleware

- It is a process of capturing and responding to errors that occur during the execution of an Express application.
- You can handle errors using middleware functions, which are functions that have access to the request and response objects, as well as the next middleware function in the application's request-response cycle.
- Example

```
const handlername = (err, req, res, next) => {
    //Code to be execute
}
app.use(handler name)
```

Logging in Node.js and Express.js

- Logging is an important part of understanding the complete application life cycle of the node.js application.
- Logging helps developers know what it is that their code is actually doing, and can help developers save hours of debugging work.
- One of the key purposes of logging is to save information about each flow or runtime.
- Node.js is supports logging libraries and packages, such as winston and morgan, offer rich features and capabilities for logging.

Logging in Node.js and Express.js

```
const winston = require("winston");
const logger = winston.createLogger({
  level: "info",  format: winston.format.json(),
  transports: [new winston.transports.Console(),
    new winston.transports.File({ filename: "logs/app.log" }),
   ],
});

app.get("/", (req, res) => {
  logger.log("info", "Request received: ", req);
  res.send("Hello, world!");
});
```

Objectives:

- In this session, our will learn to…
  - Working with Authentication and Authorization using Passport and Oauth 2.0

User Authentication with Passport.js

- Passport.js is a lightweight, flexible authentication middleware for Node.js applications.
- It allows us to incorporate user accounts and login/logout functionality into applications.
- It plays a crucial roles in securing Node.js applications by managing the authentication process.

User Authentication with Passport.js

- It ensuring that only authorized users can access protected resources from server.
- Passport achieves this by providing a standardized way to handle authentication strategies, whether it's through
  - local username/password,
  - social logins (OAuth), or
  - other methods.

Features of Passport.js

- It is easily handle success and failure.
- It is used with single sign-on with OpenID and Oauth.
- It Supports persistent sessions.
- It provides dynamic scope and permission.
- Lightweight code base.
- It also provide facility to implement custom strategies.

Installation of Passport.js

To install the passport and authentication strategies use NPM.

Use the below command on your project directory.

```
>npm install passport passport-local passport-google-oauth20
```

Middleware Configuration

1. **Set up Passport Middleware**
2. **Initialize and user Passport**

Call passport.initialize() and passport.session() to set up passport for request processing.

```
const express = require('express');
const passport = require('passport');

const app = express();

// Middleware Setup
app.use(passport.initialize());
app.use(passport.session());
```

Local Strategy Configuration

LocalStrategy is used for authenticating with a username and password stored locally

- The LocalStrategy constructor takes a **callback function** where you implement your authentication logic.
- Verify user credentials, and call **done(null, user)** if authentication is successful, or **done(null, false)** otherwise.
- Customize the callback function based on your application's user authentication mechanism.

```
const LocalStrategy =
    require('passport-local').Strategy;

passport.use(new LocalStrategy(
    (username, password, done) => {
            // Implement local
            authentication logic here
    }
));
```

OAuth Strategy Configuration

OAuth strategies are commonly used for third-party authentication, like Google OAuth

- Obtain client ID and client secret by creating a project in the Google Developer Console.
- The callback URL is where Google will redirect the user after authentication.
- The callback function is where you handle user creation or retrieval based on the Google profile information. Adjust accordingly for other OAuth providers.

```
const GoogleStrategy = require('passport-google-oauth20').Strategy;
passport.use(new GoogleStrategy({
    clientID: 'your-client-id',
    clientSecret: 'your-client-secret',
    callbackURL: 'http://localhost:3000/auth/google/callback'
},
(accessToken, refreshToken, profile, done) => {
 // Implement Google authentication logic here
}));
```

Serialization and Deserialization

Passport uses these functions to convert user data to and from the session.

- **serializeUser** defines what user data should be stored in the session. Typically, the user ID is used for this purpose.
- **deserializeUser** is responsible for retrieving the user data from the session. In this example, the user is retrieved from the database using the stored ID.
- Customize these functions based on your application's user data structure and storage mechanism.

```
passport.serializeUser((user, done) => {
  done(null, user.id);
});
passport.deserializeUser((id, done) => {
  // Retrieve user from the database based on id
  // and pass it to done
});
```

Authentication Routes

Define routes that handle the initiation of authentication processes, such as redirecting to Google OAuth.

- The authentication route redirects users to the third-party provider (Google in this example) for authentication.
- The callback route handles the response from the provider and performs actions based on successful or failed authentication.
- A logout route is included for user logout functionality. Adjust routes and logic based on your application's authentication needs.

Authentication Routes (Contd.)

```
// Google OAuth authentication route
app.get('/auth/google',
    passport.authenticate('google', { scope: ['profile', 'email'] })
);
// Google OAuth callback route
app.get('/auth/google/callback', passport.authenticate('google', { failureRedirect:
'/auth/google' }),
    (req, res) => {
        res.redirect('/');  //Successful authentication – redirect to home page
    }
);
// Logout route
app.get('/logout', (req, res) => {  req.logout();
    res.redirect('/');
});
```

Protecting Routes

Use middleware to ensure that only authenticated users can access certain routes.

- **ensureAuthenticated** is a custom middleware function that checks if the user is authenticated using **req.isAuthenticated**().
- If authenticated, it calls **next()** to allow access to the protected route; otherwise, it redirects to the login page.
- Apply the middleware to routes that require authentication, as demonstrated in the example /dashboard route. Adjust as needed for your application.

Protecting Routes **(Contd.)**

```
function ensureAuthenticated(req, res, next) {
  if (req.isAuthenticated()) {
    return next();
  }
  res.redirect('/login');
}

// Example protected route
app.get('/dashboard', ensureAuthenticated, (req, res) => {
  res.render('dashboard', { user: req.user });
});
```

Example of Protected Routes

Illustrate how to create a route that requires user authentication for access

- The /dashboard route is an example of a protected route.
- ensureAuthenticated middleware is applied to ensure that only authenticated users can access the dashboard.
- Adjust the route path and rendering logic based on your application's requirements

```
// Example protected route
app.get('/dashboard', ensureAuthenticated, (req, res) => {
  res.render('dashboard', { user: req.user });
});
```

Q&
A